SISTEMAS DE RECOMENDACIÓN · PRESENTACIÓN DE PAPER

GenRec: An LLM-Backed Recommendation Ranker at NetflixConference

Li, Sehgal, Rao, Houthooft, Hu, Zhu, Medapati, Li, Baltrunas, Huang, Rastogi y Aryafar (2026) ·
arXiv:2608.10257v2 [cs.IR]



Historia + contexto en texto → LLM · una sola pasada → **Ranking de todo el catálogo**

Presentado por: Matias De la Cuadra · 29/10/2026

00

01 Contexto

02 Problema de recomendación

03 Contribución

04 Estado del arte y marco teórico

05 Detalle de la solución

06 Evaluación

07 Conclusiones y referencias

01 · CONTEXTO

Un ranking maduro, cada vez más difícil de extender

Cómo funciona hoy

- Miles de features hechas a mano: usuario, ítem e interacciones
- Interacciones de alto orden entre features, diseñadas a mano
- Arquitecturas a medida: redes de interacción, transformers para intereses dinámicos, multi-task

Por qué se vuelve un problema

- Sumar un tipo de contenido o una superficie exige features, arquitectura, infraestructura y experimentos
- Años de iteración acumularon complejidad
- El sistema no sigue el ritmo de las necesidades de negocio

Películas · series · juegos · eventos en vivo · podcasts

Cientos de millones de miembros

01 · CONTEXTO

Los LLM abren otra vía, pero no sirven listos

La promesa

- Representar usuarios, contenido y contexto directamente en lenguaje natural
- Conocimiento del mundo y comprensión semántica de los ítems
- Adaptación rápida: guiar las recomendaciones con prompts

Un LLM genérico como recomendador

- ⚠ Sobre-recomienda contenido popular a nivel global
- ⚠ Alucina títulos que no están en el catálogo
- ⚠ Ignora restricciones de negocio
- ⚠ Personalización limitada

LA RESPUESTA DE NETFLIX

GenRec post-entrena un LLM fundacional interno con datos de Netflix y restringe su salida al catálogo.

Notación: qué entra al recomendador

U

Conjunto de usuarios

C

Catálogo: películas, series, juegos, eventos en vivo, podcasts

X

Espacio de contextos: dispositivo, superficie, locale, hora

u ∈ U, τ ∈ X, t

Una solicitud: usuario, contexto y momento

H

Historial de interacciones del usuario antes de t

M_i

Metadatos del ítem i ∈ C: título, género, sinopsis, estreno

Entrada: (u, τ, t, H) y {M_i} → Salida: ranking π del catálogo C

Qué se recomienda: un ranking del catálogo

$$(u,\ \tau,\ t,\ H) \mapsto \pi : C \to \{1, \ldots, |C|\}$$

Entrada

Usuario, contexto, momento e historial, más los metadatos M_i de cada ítem.

Salida

Un ranking π del catálogo completo; $\pi(i)$ es la posición de i (1 = tope).

Objetivo

Maximizar la utilidad esperada de largo plazo del miembro, no solo el engagement inmediato.

Caso estudiado: ranking de catálogo completo (o top-K si se entrega un conjunto candidato). El ranking sirve como representación personalizada reutilizable en varias superficies y aplicaciones.

03 · CONTRIBUCIÓN

Qué aporta el paper

01

Ranker LLM competitivo con producción

Con muchos menos datos y señales de Fase 2, supera a un ranker maduro offline y online.

02

Entrenamiento en dos fases

Fase 1: LLM abierto adaptado a Netflix. Fase 2: post-entrenamiento de ranking. Se mide cada aporte.

03

Scoring consciente del catálogo

Cabeza de ranking sobre un LLM decoder-only: puntúa todo el catálogo en una pasada, sin salirse de él.

04

Context engineering y rewards

Verbalización compacta (1/3 de los tokens) y pérdida de ranking ponderada por rewards de largo plazo.

05

Escalamiento y costo–calidad

Mejora monótona con más datos y modelos mayores; se buscan “sweet spots” en la frontera de Pareto.

RESULTADO CLAVE

+1,6 % MRR

offline, con ~40× menos ejemplos de Fase 2

GenRec frente a otros enfoques

Familia	Cómo representa y genera	Frente a GenRec
Secuenciales (SASRec, BERT4Rec)	IDs de ítems; predicen el siguiente ítem	Sin conocimiento del mundo ni prompts
Generativos con tokens especiales (HSTU, PinRec, OneRec)	Decoder con vocabulario extendido (IDs, contexto, superficie)	Poco conocimiento del mundo; steering limitado
Semantic IDs (TIGER)	Códigos RQ-VAE por ítem; decodificación autoregresiva	Ítems como códigos, no como texto
LLM + Semantic IDs (PLUM, GLIDE, OneRec-Think)	LLM con vocabulario extendido; beam search	Latencia alta con muchos candidatos
GenRec	Texto verbalizado + cabeza de scoring por catálogo; solo prefill	Un forward pass para todo el catálogo; salida solo en catálogo

Diferencia clave: el costo de inferencia. Una pasada frente a decodificación con beam search.

Recomendación generativa con tokens especiales

Secuenciales

SASRec (2018) · BERT4Rec (2019)

Predicción del siguiente ítem sobre secuencias de IDs: la plantilla base del enfoque.

Escala industrial

HSTU · PinRec · modelo fundacional de Netflix

Decoders transformer con tokens especiales para ID de ítem, metadatos, locale, tiempo, dispositivo y superficie.

Semantic IDs

TIGER (2023) · Google · Kuaishou

Cada ítem es una secuencia corta de códigos aprendidos con RQ-VAE sobre embeddings de contenido.

LIMITACIÓN QUE MOTIVA A LOS LLM

Modelos potentes, pero con poca capacidad de usar conocimiento del mundo, de guiarse con prompts flexibles o de heredar las capacidades de los modelos de lenguaje modernos.

Recomendación basada en LLM

Texto a texto (académico)

P5, TALLRec, LlamaRec: la recomendación se plantea como generación de texto con un modelo de lenguaje.

LLM con Semantic IDs (industria)

PLUM (YouTube), GLIDE (Spotify, podcasts) y OneRec-Think (Kuaishou, Qwen3 con razonamiento): vocabulario del LLM extendido con Semantic IDs.

Cuello de botella

Decodificación autoregresiva con beam search: la latencia se vuelve prohibitiva con conjuntos grandes de candidatos.

CÓMO RESPONDE GENREC

Cabeza de ranking consciente del catálogo

Rankea un conjunto grande de candidatos en un solo forward pass, y trata el contexto y los rewards como parte del diseño bajo restricciones de costo.

Conceptos clave para entender el paper

MRR

Mean Reciprocal Rank: promedio de 1/posición del primer acierto. Premia poner arriba lo que el usuario elige.

LLM decoder-only

Transformer que predice el siguiente token. Es el backbone de GenRec, heredado del LLM fundacional.

Prefill vs. decode

Prefill procesa todo el prompt en una pasada; decode genera token a token. GenRec usa solo prefill.

Post-training

Adaptar un modelo preentrenado con datos y objetivos de la tarea; aquí, en dos fases.

Context engineering

Decidir qué información entra al prompt y cómo condensarla dentro de un presupuesto de tokens.

Pérdida con rewards

Cada ejemplo se escala con un peso de modelos de reward; alternativa simple al RL completo (p. ej. GRPO).

Visión general: entrenamiento en dos fases

Fase 1 · Modelo fundacional

LLM abierto + datos propietarios de Netflix

CAPACIDADES

Conocimiento del mundo, personalización, comprensión de contenido, lenguaje

CADENCIA

Se actualiza con poca frecuencia

COSTO

Menos restringida por el costo de servir

Fase 2 · GenRec

Foco del paper: post-training para ranking

CAPACIDADES

Calidad de ranking y steering de las recomendaciones

CADENCIA

Frecuente: nuevos lanzamientos, popularidad, intereses

COSTO

Explícitamente eficiente para servir mucho tráfico

CADENCIA DE ACTUALIZACIÓN EN EL TIEMPO



Datos de post-entrenamiento: logs como diálogo

De logs a conversaciones

Cientos de miles de millones de eventos (reproducir, pulgar, agregar a la lista) en varias superficies.

Cada historial se convierte en una conversación de uno o varios turnos entre usuario y recomendador.

Formato unificado para entrenar el objetivo de ranking y el de lenguaje.

Mensaje del usuario

Contexto: superficie, hora, dispositivo, locale

Perfil e historial: país, antigüedad, plan, interacciones previas

Ítems: IDs, metadatos (título, sinopsis, estreno), tendencias de popularidad

Tarea: p. ej. predecir qué ítems reproducirá o marcará con pulgar arriba

Mensaje del asistente

Feedback real del usuario (reproducción, duración, abandono, pulgar): la señal de verdad para entrenar.

05 · DETALLE DE LA SOLUCIÓN

Verbalización e ingeniería de contexto

05

Conservar completo

Interacciones de alta señal (plays largos, pulgares arriba), con más metadatos.

Omitir

Eventos recientes de baja señal (plays muy cortos, clics ruidosos): cuestan tokens y aportan poco.

Resumir o comprimir

Conductas repetitivas, como sesiones de maratón, en vez de listar cada evento.

Elaborar selectivamente

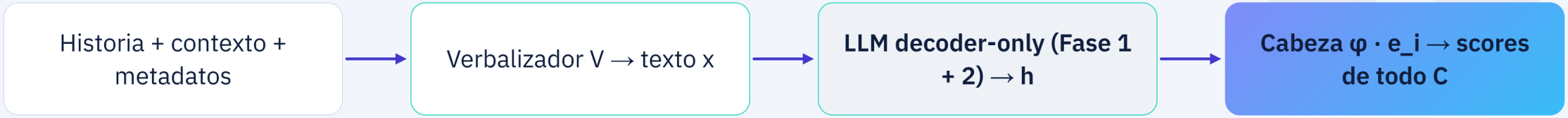
Ítems nuevos o cold-start con más metadatos, para compensar lo que el modelo no sabe.

EL PROMPT ES EL NUEVO VECTOR DE FEATURES

Más granularidad para la historia reciente; la antigua se omite o se resume en un perfil breve de intereses. Meta: pocos tokens, muy informativos.

05 · DETALLE DE LA SOLUCIÓN

Arquitectura y scoring consciente del catálogo



1 · VERBALIZACIÓN

$$x = V(H, \{M_i\}_{i \in C}, \tau)$$

Un texto único con historial, catálogo y contexto.

2 · REPRESENTACIÓN

$$h = LLM_\theta(x)$$

Estado oculto d-dimensional (posición de pooling).

3 · SCORING POR CATÁLOGO

$$s_i = \varphi(h, e_i)$$

$$p(i|x) = \mathrm{softmax}(s)$$

e_i: embedding aprendido; π ordena los s_i.

θ (pesos del LLM), φ y {e_i} se entrenan juntos. Con catálogos muy grandes se combina el scoring con sampled-softmax.

Objetivo de post-entrenamiento

$$L = \alpha \cdot L_{ranking} + \beta \cdot L_{language} + \gamma \cdot L_{misc}$$

$$\text{s.t. } \alpha + \beta + \gamma = 1,\ \alpha, \beta, \gamma \geq 0 \text{ (hiperparámetros tuneados offline)}$$

L_ranking

Entropía cruzada sobre el catálogo (o conjunto candidato). Positivos: engagements de alto valor, con denoising y umbrales por tipo de contenido.

L_language

Modelado de lenguaje sobre entradas y salidas verbalizadas (títulos, campos de texto). Conserva comprensión del lenguaje y steering por prompts.

En inferencia

Solo se usa la tarea de ranking. El lenguaje aporta señal textual y deja abierta la generación de explicaciones.

Rewards: valor a largo plazo y reglas de negocio

Proxies de satisfacción a largo plazo

Modelos de reward estiman cuánto se asocia un engagement inmediato con volver al servicio, explorar más catálogo o sostener el uso. Las métricas reales de largo plazo son ruidosas y tardías.

Rebalanceo de comportamiento

Ajuste entre tipos de contenido (películas, juegos, vivo, podcasts) y etapas de lanzamiento (pre-lanzamiento, recién lanzado, evergreen), por ejemplo para equidad de exposición.

PÉRDIDA PONDERADA POR REWARD

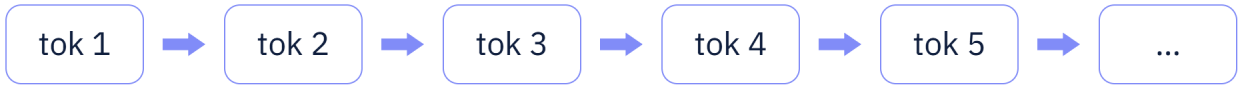
$$L_{rank} = \Sigma_n \, w_n \cdot CE_n$$

Forma ilustrativa: w_n es el peso escalar de cada ejemplo, derivado de varios modelos de reward. Alto valor pesa más; conductas no deseadas, menos.

¿Por qué no RL completo? GRPO dio ganancias extra en pruebas preliminares, pero su costo de entrenamiento lo deja para trabajo futuro.

Serving: prefill-only y control de costo

Decodificación autoregresiva
N pasadas, token a token



Prefill-only (GenRec)
1 pasada sobre el contexto

Contexto → scores de todo el catálogo

Palancas de costo: costo ≈ tamaño del modelo × longitud de contexto

Modelos más pequeños y destilados, entrenados con más datos.

Verbalización compacta: ≈⅓ de los tokens, ≈⅓ del costo.

vLLM sobre el stack de serving LLM interno de Netflix.

06 · EVALUACIÓN

GenRec frente al ranker en producción

~40×

menos ejemplos etiquetados de Fase 2 que el modelo de producción

+1,6 %

MRR relativo offline sobre el baseline

10 %

del tráfico durante 4 semanas en el A/B test

+0,006 %

relativo en la métrica online central, estadísticamente significativo

Baseline: modelo discriminativo maduro, afinado durante años. GenRec se probó con pocos datos y pocas señales, en superficies de cómputo batch.

Mejoras significativas en métricas online de corto y largo plazo (Figura 3 del paper).

Escalamiento de datos y de modelo

Más datos de Fase 2

1× → 20×

Con modelos de ~1B y ~10B parámetros, el MRR offline mejora de forma monótona. El de ~1B parte más bajo, pero sigue el mismo patrón.

Backbones más grandes

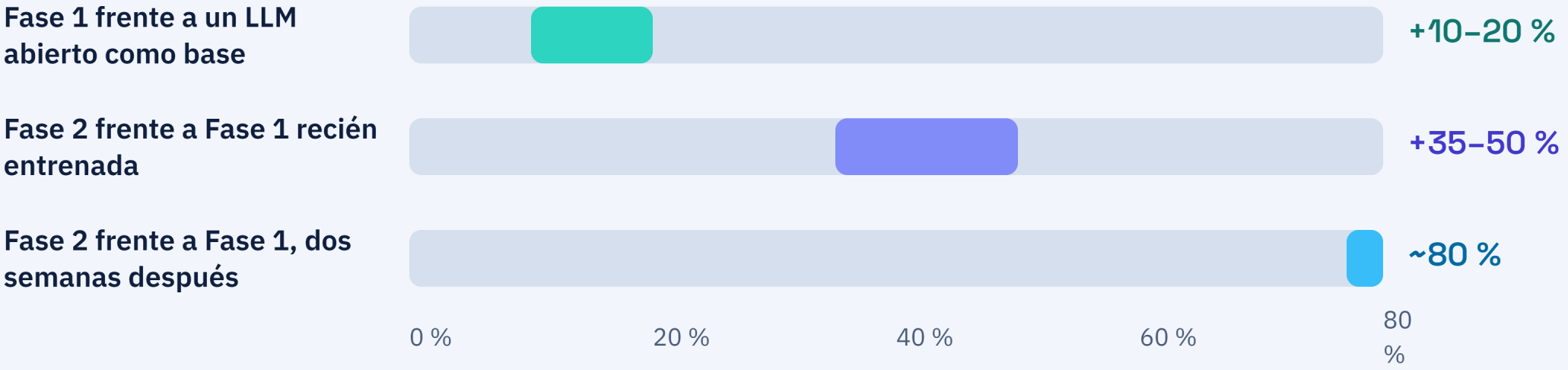
~1B → ~10B

Con presupuesto de entrenamiento fijo (misma configuración de GPU, tiempo similar), los modelos mayores logran MRR offline más alto.

TRADE-OFF CALIDAD–COSTO

Con las curvas de datos, de modelo y de contexto se elige un “sweet spot” en la frontera de Pareto: la mayor parte de la mejora sin costos prohibitivos de entrenamiento y serving.

Aporte de cada fase en MRR offline



El beneficio de la Fase 2 crece con el tiempo porque el backbone de la Fase 1, que se actualiza poco, se desactualiza frente a nuevos lanzamientos, popularidad e intereses.

Optimización de la longitud de contexto

1 · Selección y compresión

Se omiten eventos ruidosos, se comprime lo repetitivo y se retienen las interacciones de alta señal.

2 · Barrido de la historia

MRR frente al número de eventos: hay un punto de codo y más tokens dan ganancias marginales (Figura 5).

3 · Verbalización concisa

Se prueban niveles de detalle por evento, redacción más simple y prompts sin ejemplos few-shot.

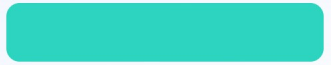
TOKENS DE CONTEXTO

Original



~5.000

Optimizado



~1.700

≈ ⅓ del costo de serving

Degradación despreciable del MRR offline

Qué cambia con un recomendador LLM

GenRec es un primer paso hacia un stack de recomendación centrado en LLM.

Features → contexto: el prompt es el nuevo vector de features.

Arquitecturas → backbones: la innovación sube a datos, scaling y post-training.

Scaling laws como guía de diseño, no como observación posterior.

Pre-training, post-training y rewards adaptan un backbone compartido.

Infra RecSys → infra LLM: KV-cache, prefix caching, prefill-only, vLLM.

Mirada crítica

Evaluado solo en superficies de cómputo batch.

Mejora online relativa pequeña (+0,006 %), aunque significativa.

LLM fundacional y datos propietarios: difícil de reproducir.

El costo de serving sigue siendo la restricción central.

Referencias principales

Secuencial y generativa

Kang y McAuley (2018). Self-attentive sequential recommendation (SASRec). arXiv:1808.09781.

Sun et al. (2019). BERT4Rec. CIKM ’19.

Rajput et al. (2023). Recommender systems with generative retrieval (TIGER). NeurIPS ’23.

Zhai et al. (2024). Actions speak louder than words (HSTU). arXiv:2402.17152.

Singh et al. (2024). Better generalization with Semantic IDs. RecSys ’24.

Deng et al. (2025). OneRec: unifying retrieve and rank. arXiv:2502.18965.

LLM, rewards y contexto

Geng et al. (2022). Recommendation as language processing (P5). RecSys ’22.

Bao et al. (2023). TALLRec. RecSys ’23.

He et al. (2026). PLUM: adapting pre-trained language models for industrial-scale generative recommendations. WWW ’26.

D’Amico et al. (2026). Semantic ID-based generative retrieval for podcast discovery at Spotify (GLIDE). arXiv:2603.17540.

Liu et al. (2025). OneRec-Think. arXiv:2510.11639.

Tang et al. (2023). Reward innovation for long-term member satisfaction. RecSys ’23.

Paper presentado: Li et al. (2026). GenRec: An LLM-Backed Recommendation Ranker at Netflix. arXiv:2608.10257v2.